HW 4.3: A First QMD File

Nihat Hasan

2026-04-14

Busiest Airports Analysis

This section compares passenger traffic for six major airports from 2020 through 2025. Table 1 provides the exact passenger counts, while Figure 1 shows broader trends over time.

Table 1: Passenger traffic for six major airports from 2020 through 2025.

Airport	2020	2021	2022	2023	2024	2025
Beijing Daxing International Airport	NA	25,051,012	NA	NA	49,441,029	53,618,949
Frankfurt Airport	18,768,601	24,812,849	48,918,482	59,355,389	61,564,957	63,189,666
Hartsfield-Jackson Atlanta International Airport	42,918,685	75,704,760	93,699,630	104,653,451	108,067,766	106,302,208
Istanbul Airport	23,330,411	36,988,563	64,289,107	76,027,321	80,073,252	84,457,421
John F. Kennedy International Airport	16,630,642	30,788,322	55,287,693	62,464,331	63,265,984	62,629,455
Los Angeles International Airport	28,779,527	48,007,284	65,924,298	75,050,875	76,588,028	73,709,594

The table in Table 1 and the line graph in Figure 1 work together to show both exact passenger totals and overall patterns. One clear result is that Hartsfield-Jackson Atlanta International Airport remained the busiest airport in this group throughout the period. The figure makes Atlanta's lead visually obvious, while the table allows comparison of exact differences between airports and years. Another important pattern is the strong rebound after 2020, with most airports showing growth over time. Istanbul Airport stands out because it saw sharp growth and finished second among the six airports by 2025. Frankfurt, JFK, and LAX also rose over time, even though they stayed below Istanbul and Atlanta. Beijing Daxing International Airport has missing values in some years, so its trend is less complete, but the available observations still show growth. Overall, the table is most useful for precise comparison, while the figure is better for seeing long-term changes and relative ranking.

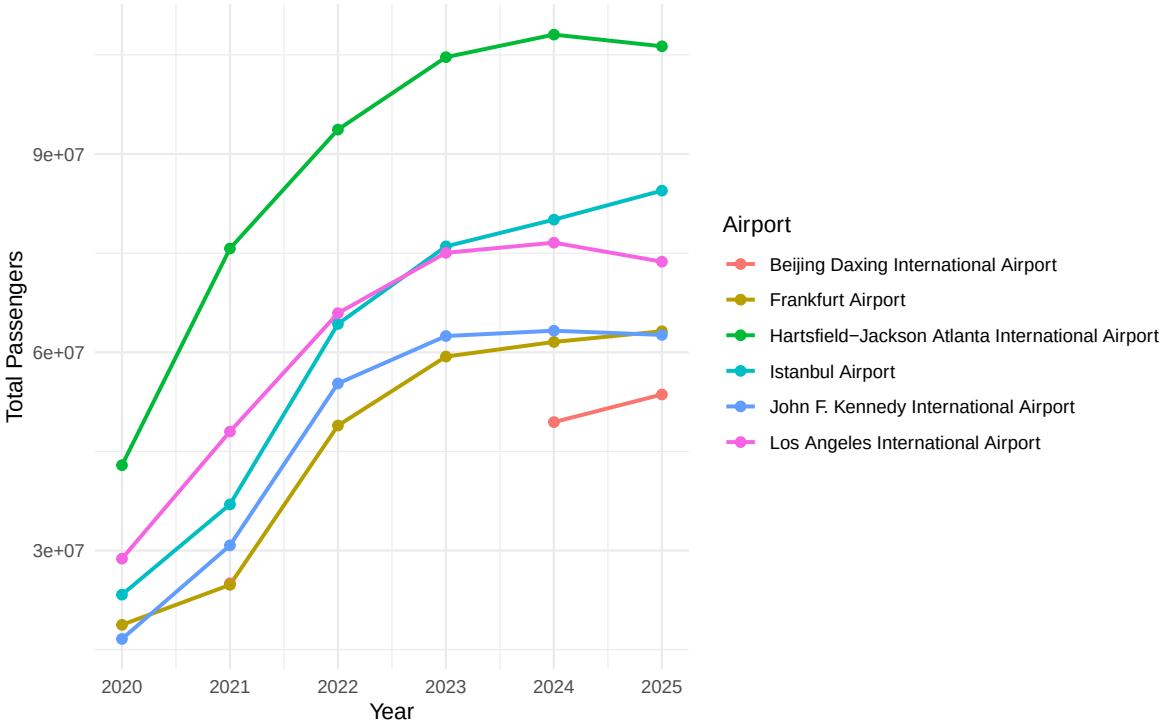
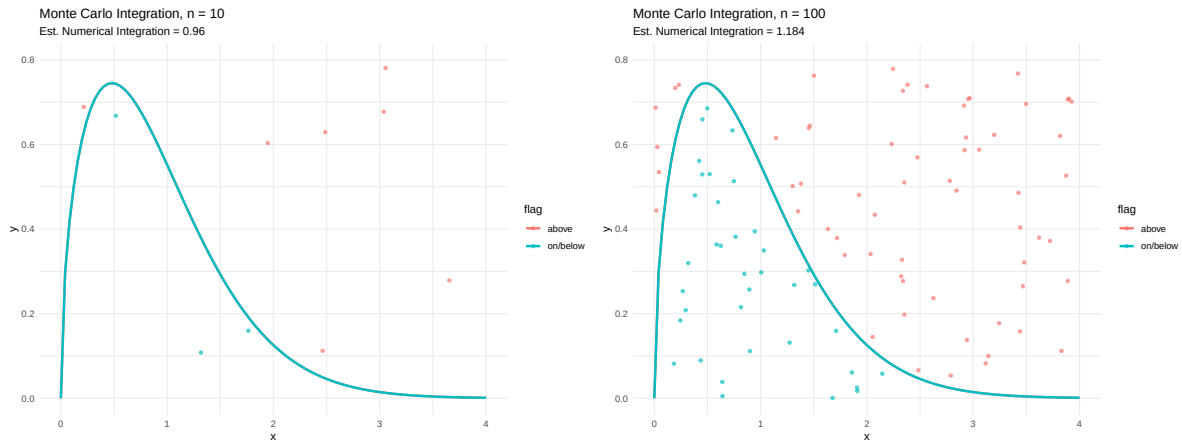


Figure 1: Passenger traffic trends for six major airports from 2020 through 2025.

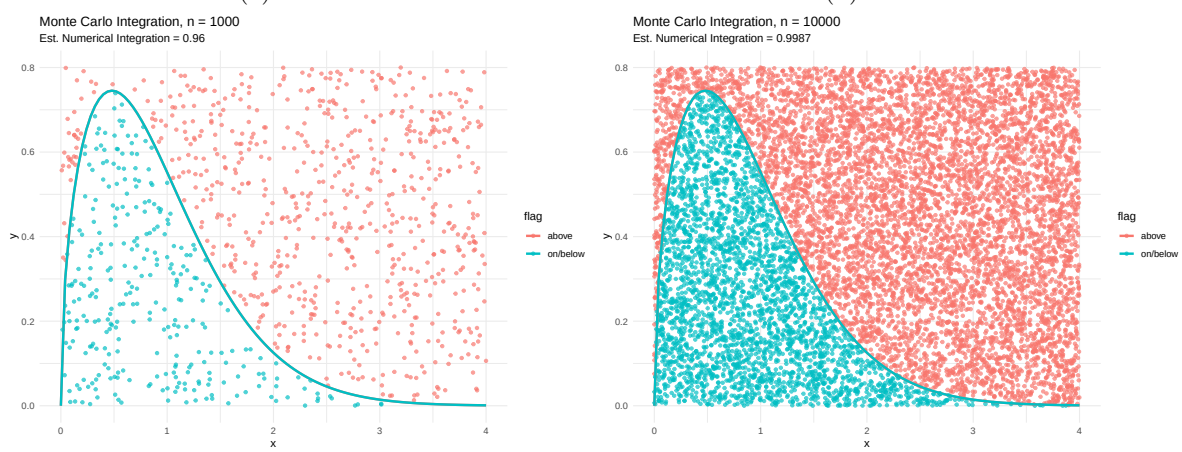
Monte Carlo Numerical Integration

This section uses a Monte Carlo simulation to estimate the integral of the assigned Weibull density over the region 0 x 4 and 0 y 0.8. Figure 2 shows the results at four different resolutions.



(a) n = 10

(b) n = 100



(c) n = 1000

(d) n = 10000

Figure 2: Monte Carlo numerical integration for the Weibull density at four resolutions.

Monte Carlo integration estimates the area by sampling random points inside a rectangle and checking whether each point lies above, on, or below the curve. The proportion of points on or below the curve is then multiplied by the area of the rectangle to estimate the integral. In this case, the estimates in Figure Figure 2 become more stable as the number of simulated points increases. I think the exact answer is very close to 1. I reason that the function being integrated is a probability density function, so the total area over a sufficiently large region should be close to 1, and estimates at higher resolutions approach that value. The denser panels provide

stronger support because they reduce random fluctuation and make the simulated proportion more stable.

Planning and Prompting GenAI Tools

Data Context

The calcium data come from a longitudinal study of 31 women measured across four time points. The first four columns represent the null treatment group, and the second four columns represent the diet and exercise treatment group. Because repeated measurements are spread across multiple columns, the data are not tidy in their original form.

Plan

Goals

My first goal is to tidy the calcium data so that each row represents a woman at a single time point. My second goal is to label the treatment groups and time points clearly so that others can interpret the data correctly. My third goal is to create a plot that compares ulnar calcium measurements over time for the null treatment group and the diet-and-exercise group. I will need the calcium.csv file, the treatment group labels, the time-point labels, a tidy data frame, the data wrangling code, and a plot that allows comparison across both time and treatment group.

Needs

I will need the calcium.csv file, the treatment group labels, the time-point labels, a tidy data frame, the data wrangling code, and a plot that allows comparison across both time and treatment group.

Steps

1. Read in the calcium.csv file without changing the original file.
2. Identify that the first four columns belong to the null treatment group and the second four columns belong to the diet and exercise group.
3. Relabel the columns so the time points are clear: Initial, Year 1, Year 2, and Year 3.
4. Add a woman identifier within each treatment group.
5. Reshape the data from wide to long, tidy format.
6. Create variables for the treatment group, time, and calcium measurement.

7. Remove rows with missing calcium values only when needed for plotting or summaries.
8. Create a plot comparing calcium measurements over time for the two treatment groups.
9. Use readable labels, a clear title, and an appropriate geometry for repeated measurements over time.

Plan-Informed ChatGPT Response

I used ChatGPT with the original calcium.csv file and a plan-informed prompt. The response explained that the data should be converted from wide to long format, with variables for woman identifier, treatment, time, and ulnar calcium. It then provided R code using dplyr, tidyr, and ggplot2 to reshape the data and create a line plot of group means with error bars. This response was more useful because it recognized the study design, the treatment groups, and the repeated time points.

Generic ChatGPT Response

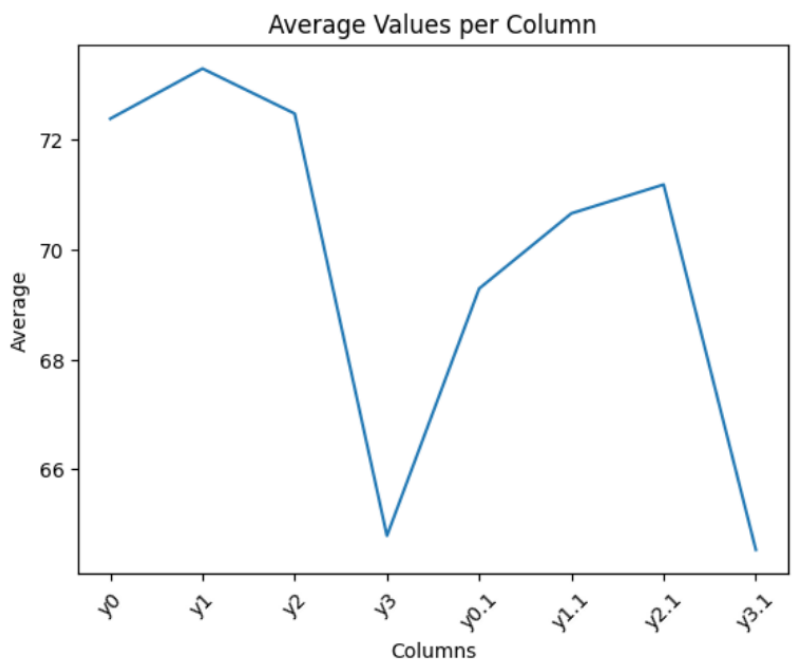


Figure 3: Generic ChatGPT output for the calcium data using an unspecific prompt.

The generic ChatGPT response used the prompt "Create a data visualization for these data. Calcium.csv." It produced a graph of the average value of each raw column and described the points as averages for separate variables. This response stayed at the raw-column level and did not reorganize the data into a tidy structure.

Comparison

The plan-informed and generic ChatGPT results were noticeably different in both usefulness and awareness of the data context. The plan-informed version recognized that the data were not tidy, identified the two treatment groups, described the four time points, and proposed a tidy long-format structure with a woman identifier. Because of that, the generated code and graph were designed to answer the actual question of how ulnar calcium changes over time for the two groups. By contrast, the generic response treated the columns as separate variables and created a plot of column averages. While that graph was not meaningless, it did not make the repeated-measures structure or the treatment comparison clear. This comparison shows that a plan-informed prompt can yield a response that is much more aligned with the context and analytical goal. In contrast, a generic prompt is more likely to yield a superficial result that overlooks important features of the data.

Reflection

So far in this course, one of the biggest things I have learned is that making a data visualization is not just about producing something that looks correct. It is also about aligning the data structure, the question being asked, and the display's visual form. Earlier in the semester, I focused more on whether I could make a graph in R, but now I think more carefully about whether the graph helps someone learn. For example, in the busiest airports section, I learned that a table and a plot serve different purposes: the table supports exact comparison, while the plot highlights larger patterns over time. In the Monte Carlo section, I learned that a visualization can also support reasoning about a mathematical process, not just display data. The small multiple helped me see how increasing the number of sampled points made the estimate more stable. I also learned that tidy data matters because it makes later wrangling and plotting much easier. The calcium example, especially, reinforced that point, since the original wide format obscured the study's structure. In conclusion, I have become better at integrating data structures, wrangling, visualization, and interpretation, rather than treating them as separate tasks.

Appendix A: GenAI Usage

Usage 1: Planning and Prompting GenAI Section

Tool: ChatGPT Version information from generic response document: GPT-5.3 Date of plan-informed usage: April 14, 2026

Prompt:

Using the attached calcium.csv file, help me tidy and wrangle the data in R and then create a plot that helps compare ulnar calcium measurements across time for the null treatment group and the diet/exercise treatment group. The data are not tidy. The first four columns belong to the null treatment group and represent Initial, Year 1, Year 2, and Year 3. The second four columns belong to the diet/exercise group and represent Initial, Year 1, Year 2, and Year 3. Please do all of the following: explain briefly how the data should be tidied, write R code to turn the data into tidy long format, include a woman identifier, handle missing values appropriately, and create an appropriate ggplot visualization for comparing the two groups across time. Please also explain why your chosen plot is appropriate for this context.

Exact response text:

Here is a clean way to do it.

How the data should be tidied

Right now, the data are in wide format: each woman's repeated measurements are spread across four separate columns for each treatment group.

A tidy version should have one row per observation, with these variables:

woman_id treatment time ulnar_calcium

So instead of one row holding all four years, each woman will have up to four rows, one for each time point.

A good way to handle the missing values here is to keep them as NA in the tidy dataset and then remove them only when plotting or computing summaries. In this file, the null group has one row that is entirely missing, so it should not be imputed.

R code library(dplyr) library(tidyr) library(ggplot2)

read the file calcium <- read.csv("calcium.csv")

build tidy data for the null group null_group <- calcium %>% transmute(woman_id = paste0("N", row_number()), treatment = "Null", Initial = y0, `Year 1` = y1, `Year 2` = y2, `Year 3` = y3)

build tidy data for the diet/exercise group diet_group <- calcium %>% transmute(woman_id = paste0("D", row_number()), treatment = "Diet/Exercise", Initial = y0.1, `Year 1` = y1.1, `Year 2` = y2.1, `Year 3` = y3.1)

combine both groups and pivot to long format calcium_tidy <- bind_rows(null_group, diet_group) %>% pivot_longer(cols = c(Initial, `Year 1`, `Year 2`, `Year 3`), names_to = "time", values_to = "ulnar_calcium") %>% mutate(time = factor(time, levels = c("Initial", "Year 1", "Year 2", "Year 3")))

view tidy data calcium_tidy

remove missing calcium values only for plotting/summaries calcium_plot <- calcium_tidy %>% filter(!is.na(ulnar_calcium))

compute group means and standard errors at each time point calcium_summary <- calcium_plot %>% group_by(treatment, time) %>% summarise(n = n(), mean_calcium = mean(ulnar_calcium), se = sd(ulnar_calcium) / sqrt(n), .groups = "drop")

plot ggplot(calcium_summary, aes(x = time, y = mean_calcium, group = treatment, color = treatment)) + geom_line(linewidth = 1) + geom_point(size = 3) + geom_errorbar(aes(ymin = mean_calcium - se, ymax = mean_calcium + se), width = 0.1) + labs(title = "Average Ulnar Calcium Over Time by Treatment Group", x = "Time", y = "Ulnar calcium measurement", color = "Treatment") + theme_minimal() Why this plot is appropriate

A line plot is a good choice because time is ordered, and the main goal is to compare how the two treatment groups change from Initial to Year 3.

Using one line for each treatment group makes it easy to see:

the overall trend over time whether one group tends to be higher than the other how much the average changes by Year 3

The error bars add useful information about variability at each time point, so the graph shows more than just the averages.

A brief write-up you could use

The original data were not tidy because repeated measurements across time were stored in separate columns for each treatment group. To tidy the data, I created a woman identifier for each subject, added a treatment variable, and reshaped the four time columns into a single time column with one corresponding ulnar_calcium value column. Missing values were kept as NA in the tidy dataset and removed only for plotting and summary calculations. I used a line plot of group means across time with error bars because time is an ordered variable and the goal is to compare changes in ulnar calcium over time between the null and diet/exercise groups.

Usage 2: Generic ChatGPT Response Provided in MOM

Tool: ChatGPT Version information from response document: GPT-5.3 Date from response document: Tuesday, March 31, 2026

Prompt:

Create a data visualization for these data. Calcium.csv

Exact response text:

I created a visualization showing the average value of each column in your dataset.

What this shows:

Each point represents the mean (average) of one column (y0, y1, y2, y3, etc.).

You can quickly compare which variables tend to have higher or lower values overall.

For example, some columns cluster around ~70–73, while others drop closer to ~65.

If you want something more insightful, I can also create:

Boxplots (to show spread and outliers)

Time series plots (if rows represent time/order)

Correlation heatmaps

Side-by-side comparisons (e.g., y0 vs y0.1)

Just tell me

Prompt:

What is your version information?

Exact response text:

You're chatting with ChatGPT based on the GPT-5.3 architecture.

Key details:

Model: GPT-5.3

Knowledge cutoff: August 2025

Current date (system): March 31, 2026

If you need more technical details (capabilities, differences vs GPT-4/5, etc.), just let me know

(This chat took place on Tuesday, March 31st, 2026.)

Usage 3: Alt Text and Long Description Help

Tool: ChatGPT
Date: April 2, 2026

I used ChatGPT earlier in the homework process to help draft alt text and long descriptions for several visualizations, including the busiest airports graph and the Monte Carlo small multiple. I reviewed the wording before submitting those descriptions.

Appendix B: Code Appendix

Busiest Airports Code

```
library(ggplot2)
library(knitr)

# Create the airports data set used in the report.
airports_tidy <- data.frame(
  year = c(2020:2025, 2020:2025, 2020:2025, 2020:2025, 2020:2025, 2020:2025),
  iata = c(rep("PKX", 6), rep("FRA", 6), rep("ATL", 6), rep("IST", 6), rep("JFK", 6), rep("LA
  airport = c(
    rep("Beijing Daxing International Airport", 6),
    rep("Frankfurt Airport", 6),
    rep("Hartsfield-Jackson Atlanta International Airport", 6),
    rep("Istanbul Airport", 6),
    rep("John F. Kennedy International Airport", 6),
    rep("Los Angeles International Airport", 6)
  ),
  total_passengers = c(
    NA, 25051012, NA, NA, 49441029, 53618949,
    18768601, 24812849, 48918482, 59355389, 61564957, 63189666,
    42918685, 75704760, 93699630, 104653451, 108067766, 106302208,
    23330411, 36988563, 64289107, 76027321, 80073252, 84457421,
    16630642, 30788322, 55287693, 62464331, 63265984, 62629455,
    28779527, 48007284, 65924298, 75050875, 76588028, 73709594
  )
)

# Reshape the airport data into a wide table for display.
table_data <- airports_tidy[, c("airport", "year", "total_passengers")]

airport_table <- reshape(
  table_data,
  idvar = "airport",
  timevar = "year",
  direction = "wide"
)

names(airport_table) <- sub("total_passengers\\.", "", names(airport_table))
names(airport_table)[1] <- "Airport"
```

```r
airport_table <- airport_table[order(airport_table$Airport), ]
rownames(airport_table) <- NULL

for (j in 2:ncol(airport_table)) {
  airport_table[[j]] <- ifelse(
    is.na(airport_table[[j]]),
    "NA",
    format(airport_table[[j]], big.mark = ",", scientific = FALSE, trim = TRUE)
  )
}

# Create the line graph for the six airports.
airport_plot <- ggplot(
  airports_tidy,
  aes(x = year, y = total_passengers, color = airport, group = airport)
) +
  geom_line(linewidth = 0.9, na.rm = TRUE) +
  geom_point(size = 2, na.rm = TRUE) +
  scale_x_continuous(breaks = 2020:2025) +
  labs(
    x = "Year",
    y = "Total Passengers",
    color = "Airport"
  ) +
  theme_minimal()

# Display the airport table in the document.
kable(airport_table, booktabs = TRUE, align = c("l", "r", "r", "r", "r", "r", "r"))

# Display the airport plot in the document.
airport_plot
```

Monte Carlo Code

```r
set.seed(184)

# Generate random x and y coordinates for a Monte Carlo simulation.
monte_carlo_sim <- function(n, x_min, x_max, y_min, y_max) {
  if (x_min >= x_max || y_min >= y_max) {
    stop("Each minimum bound must be less than its maximum bound.")
```

```r
  }

  x_vals <- runif(n, min = x_min, max = x_max)
  y_vals <- runif(n, min = y_min, max = y_max)

  data.frame(
    x = x_vals,
    y = y_vals
  )
}

# Set the rectangle bounds for the Weibull density problem.
x_min <- 0
x_max <- 4
y_min <- 0
y_max <- 0.8

# Create a reusable function that simulates points, estimates the integral,
# and builds a plot for a chosen sample size.
make_mc_plot <- function(n) {
  mc_data <- monte_carlo_sim(n, x_min, x_max, y_min, y_max)

  mc_data$fx <- dweibull(mc_data$x, shape = 1.5, scale = 1)
  mc_data$flag <- ifelse(mc_data$y <= mc_data$fx, "on/below", "above")

  rect_area <- (x_max - x_min) * (y_max - y_min)
  p_hat <- mean(mc_data$flag == "on/below")
  est_integral <- rect_area * p_hat

  ggplot(mc_data, aes(x = x, y = y, color = flag)) +
    geom_point(alpha = 0.7, size = 1.2) +
    stat_function(
      fun = dweibull,
      args = list(shape = 1.5, scale = 1),
      xlim = c(0, 4),
      linewidth = 1.1
    ) +
    coord_cartesian(xlim = c(0, 4), ylim = c(0, 0.8)) +
    labs(
      title = paste("Monte Carlo Integration, n =", n),
      subtitle = paste("Est. Numerical Integration =", round(est_integral, 4)),
      x = "x",
```

```r
      y = "y",
      color = "flag"
    ) +
    theme_minimal()
}

# Build the four plots used in the small multiple.
plot10 <- make_mc_plot(10)
plot100 <- make_mc_plot(100)
plot1000 <- make_mc_plot(1000)
plot10000 <- make_mc_plot(10000)

# Display the four plots in Quarto's multi-pane layout.
plot10
plot100
plot1000
plot10000
```

Planning and Prompting GenAI Code

```r
# Display the generic ChatGPT image in the document.
knitr::include_graphics("chatgptPlot.png")
```